DeepLearning.AI

Model Serving

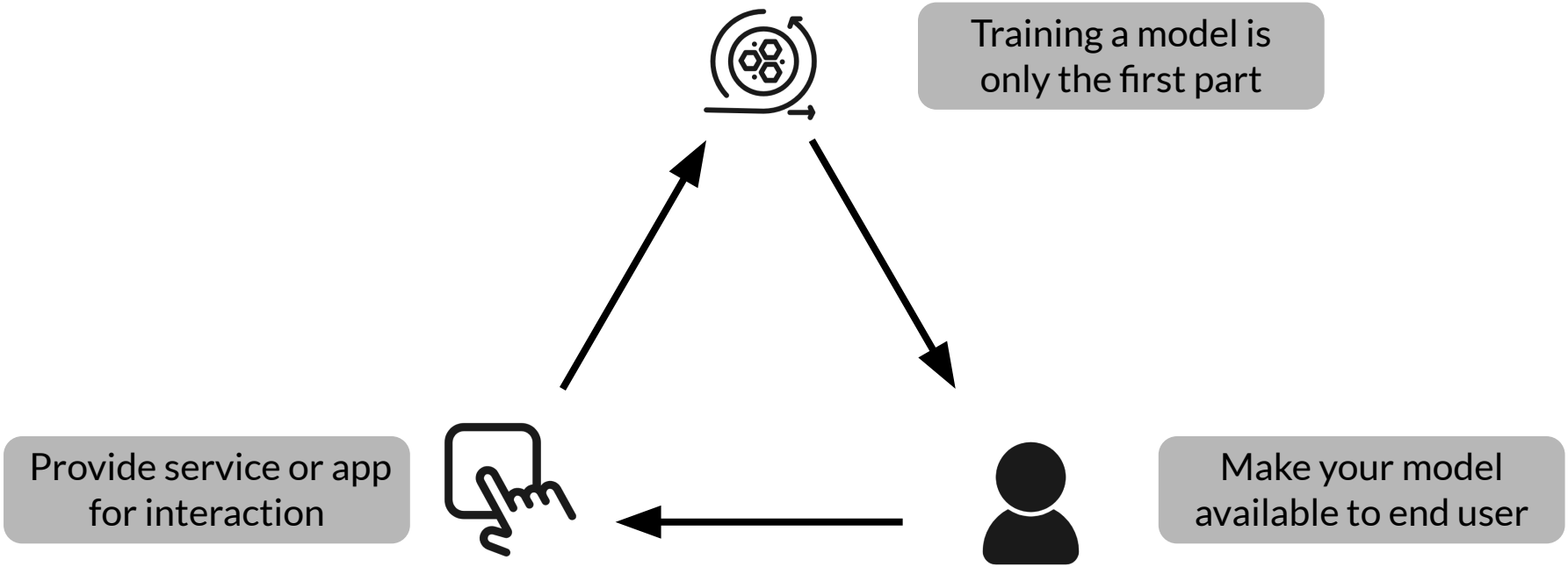
Welcome

DeepLearning.AI

Introduction to Model Serving

Introduction

What exactly is Serving a Model?



Model Serving Patterns

- A model,
- An interpreter, and
- Input data

Inference

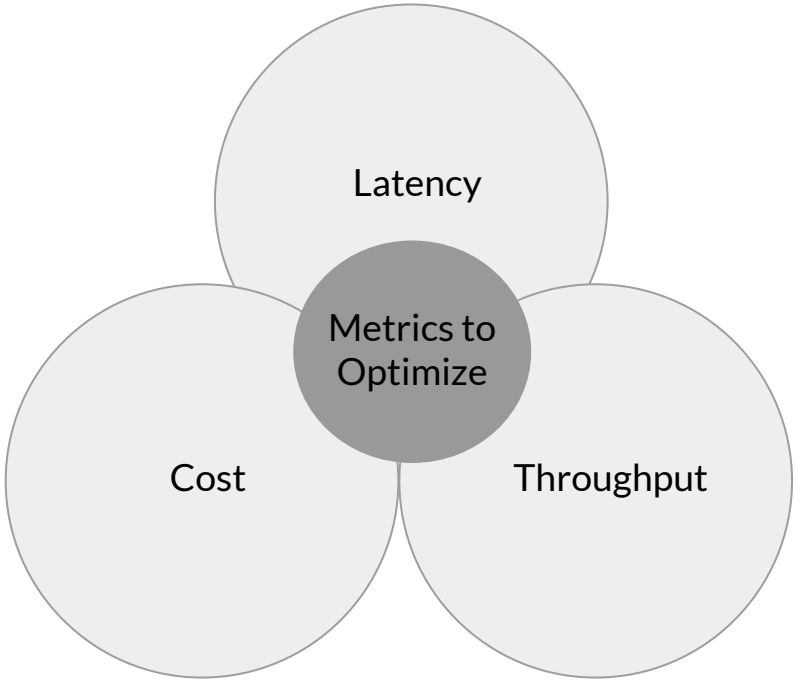
ML workflows

- Model training
- Model prediction

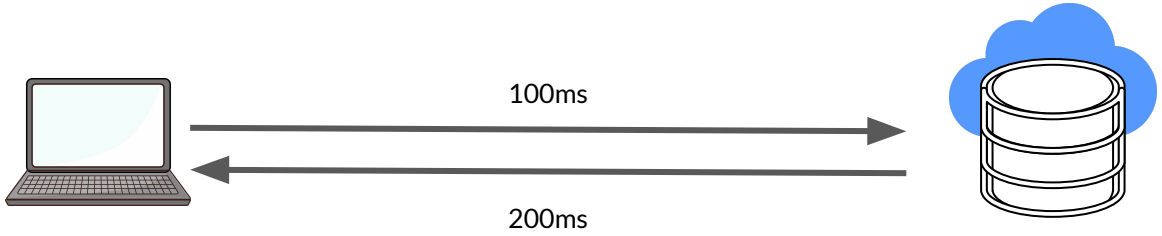
Batch inference

Realtime inference

Important Metrics



Latency



Latency = 100 + 200 = 300ms

- Delay between user's action and response of application to user's action.
- Latency of the whole process, starting from sending data to server, performing inference using model and returning response.
- Minimal latency is a key requirement to maintain customer satisfaction.

Throughput

- ➔ Throughput -> Number of successful requests served per unit time say one second.
- ➔ In some applications only throughput is important and not latency.

Cost

- The cost associated with each inference should be minimised.
  - Important Infrastructure requirements that are expensive:
    - CPU
    - Hardware Accelerators like GPU
    - Caching infrastructure for faster data retrieval.

Minimizing Latency, Maximizing Throughput

Minimizing Latency	• Airline Recommendation Service • Reduce latency for user satisfaction
Maximizing Throughput	• Airline recommendation service faces high load of inference requests per second.

Scale infrastructure (number of servers, caching requirements etc.) to meet requirements.

Balance Cost, Latency and Throughput

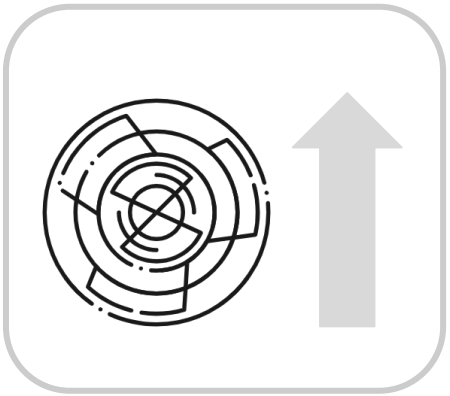
- Cost increases as infrastructure is scaled
- In applications where latency and throughput can suffer slightly:
  - Reduce costs by GPU sharing
  - Multi-model serving etc.,
  - Optimizing models used for inference

DeepLearning.AI

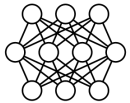
Introduction

Resources and Requirements for Serving Models

Optimizing Models for Serving



Model Complexity

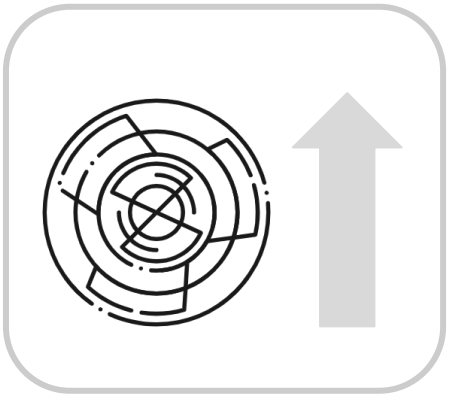


Model Size
Complex functions

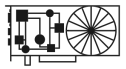
Prediction Latency

Prediction Accuracy

As Model Complexity Increases Cost Increases



Model Complexity



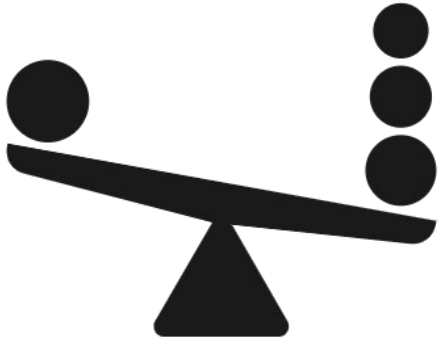
GPU
TPU

Model
registry

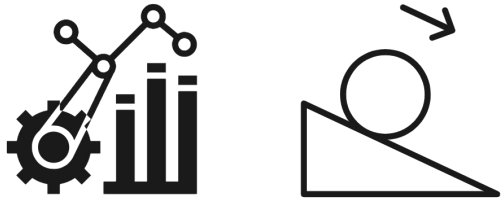
Maintenance
burden

Balancing Cost and Complexity

The challenge for ML practitioners is to balance complexity and cost.

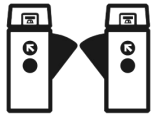


Optimizing and Satisficing Metrics



Model's optimizing metric:

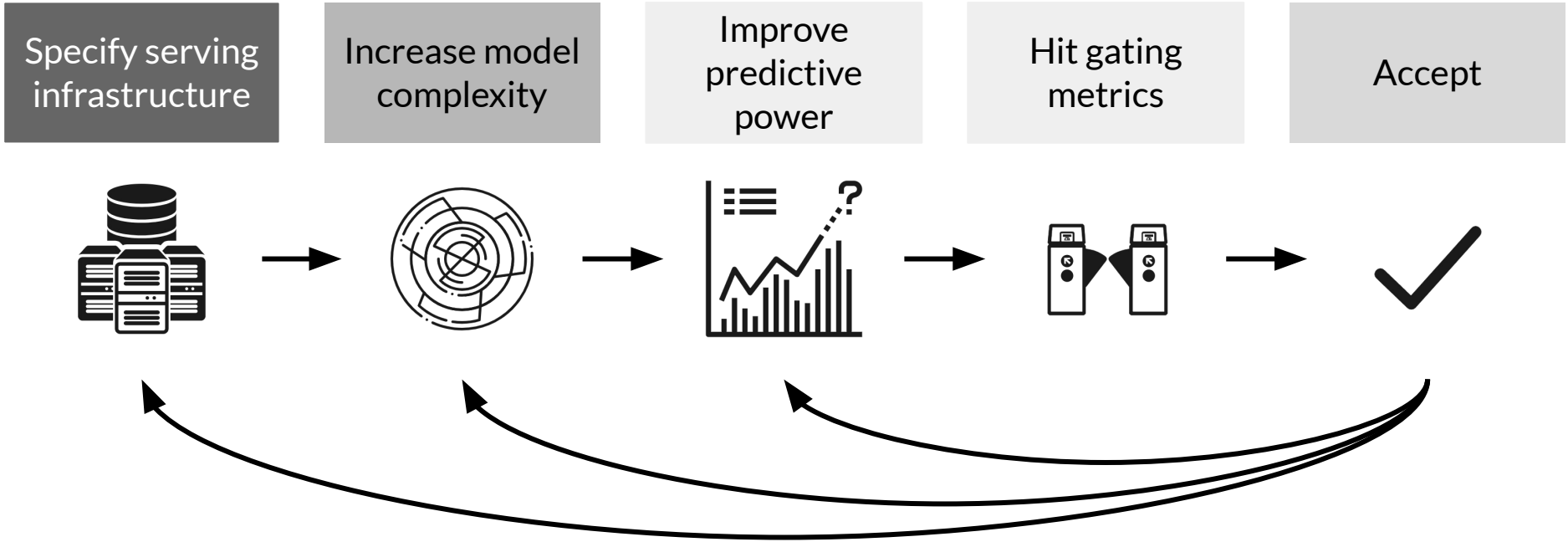
- Accuracy
- Precision
- Recall



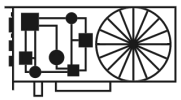
Satisficing (Gating) metric:

- Latency
- Model Size
- GPU load

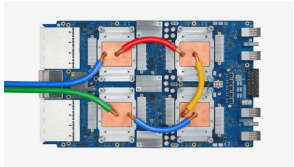
Optimizing and Satisficing Metrics



Use of Accelerators in Serving Infrastructure



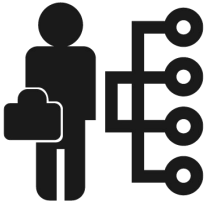
GPUs for parallel throughput



TPUs for complex models and large batches

Hardware choices impact cost

Balancing complexity and hardware choices



Choices made at organizational level

Maintaining Input Feature Lookup

- Prediction request to your ML model might not provide all features required for prediction
- For example, estimating how long food delivery will require accessing features from a data store:
    - Incoming orders (not included in request)
    - Outstanding orders per minute in the past hour
- Additional pre-computed or aggregated features might be read in real-time from a data store
- Providing that data store is a cost

NoSQL Databases: Caching and Feature Lookup

NoSQL Databases

Google Cloud Memorystore
In memory cache, sub-millisecond read latency

Google Cloud Firestore
Scaleable, can handle slowly changing data, millisecond read latency

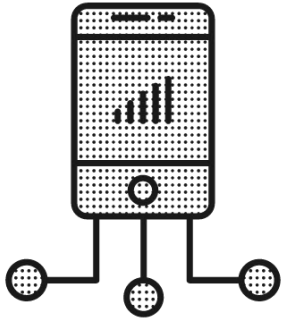
Google Cloud Bigtable
Scaleable, handles dynamically changing data, millisecond read latency

Amazon DynamoDB
Single digit millisecond read latency, in memory cache available

Expensive.
Carefully choose caching requirements

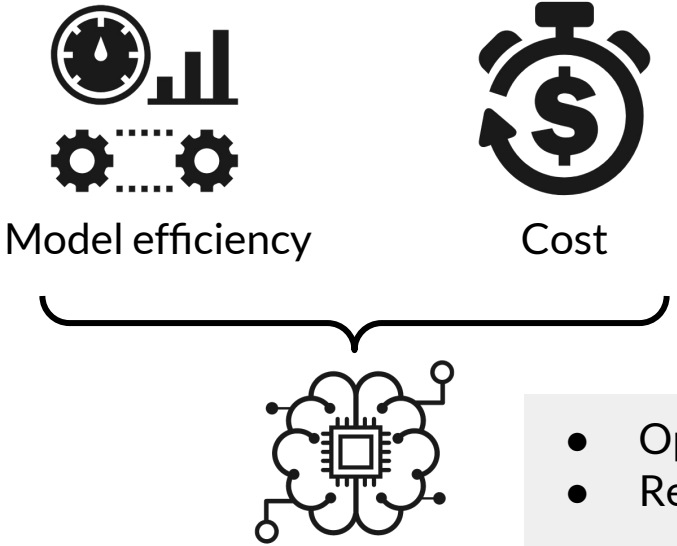
Model Deployments

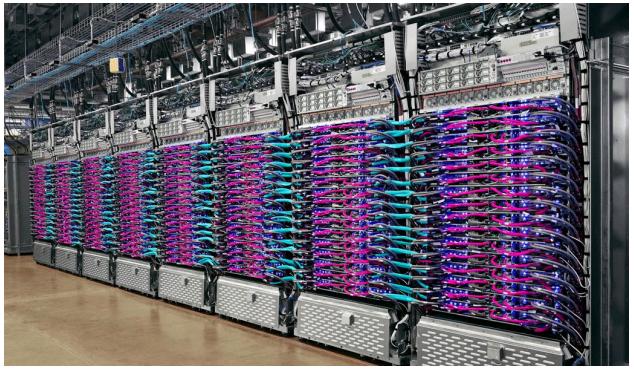
- Huge data centers



- Embedded devices

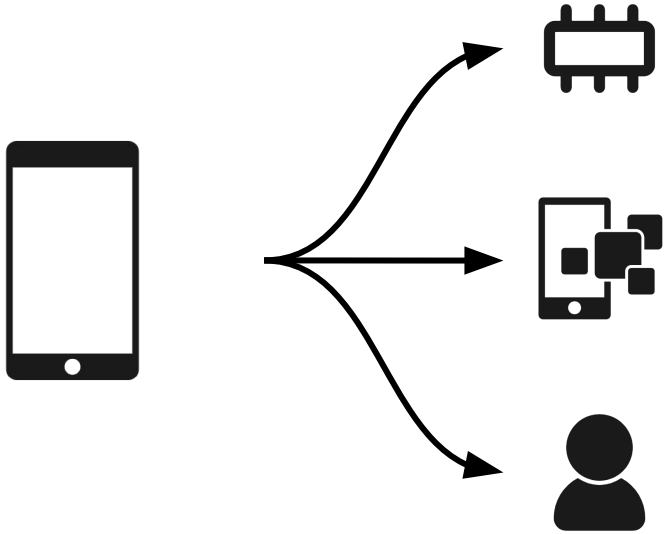
Running in Huge Data Centers





- Optimize resource utilization
- Reduce cost

Constrained Environment: Mobile Phone

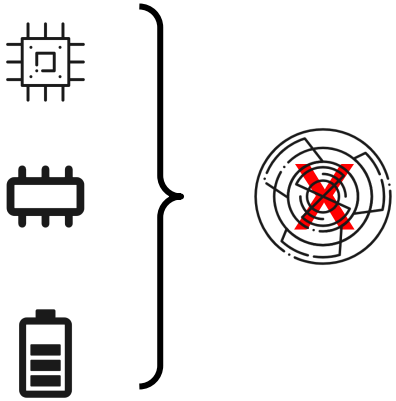


Average GPU memory size < 4GB.

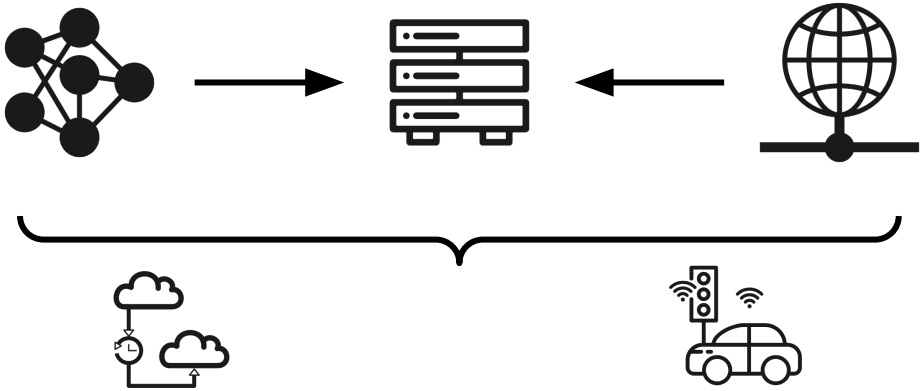
Average Android app size ≅ 11MB

Users might not install your app if it uses too much storage

Restrictions in a Constrained Environment



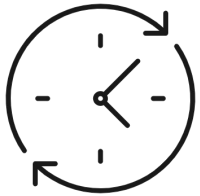
Large, complex models cannot be deployed to edge devices



Will not work when prediction latency is important. E.g. autonomous car.

Prediction Latency is Almost Always Important

- Opt for on-device inference whenever possible
  - Enhances user experience by reducing the response time of your app



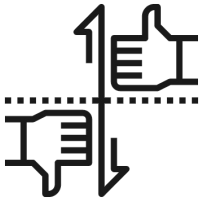
Millisecond turnaround

Model efficiency

Cost

Choose Best Model for the Task

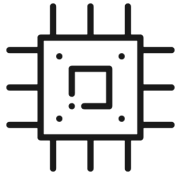
Other Strategies



Profile and Benchmark

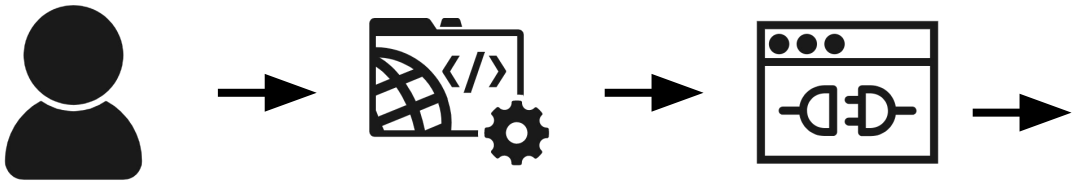
Optimize Operators

Optimize Model



Tweak Threads

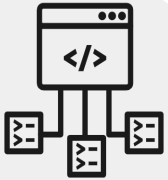
Web Applications for Users



Users make requests via web application

Model wrapped as API service

Python:
- FastAPI
- Flask
- Django



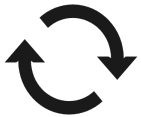
Java:
- Spring
- Apache Tomcat

Serving systems for easy deployment

- Centralized model deployment
- Predictions as service

Eliminates need for custom web applications

Deployment just a few lines of code away

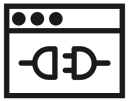


Easy to rollback/update models on the fly

Clipper

Open-source project from UC Berkeley

Multiple modeling frameworks



RESTful API

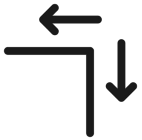
Cluster and resources management

Settings for reliable latency

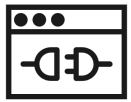
TensorFlow Serving

Open-source project from Google

Serve TensorFlow models easily



Extensible to serve other model types



Uses REST and gRPC protocol

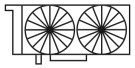
Version manager

Advantages of Serving with a Managed Service

Realtime endpoint for low-latency predictions on massive batches

Deployment of models trained on premises or on the Google Cloud Platform

Scale automatically based on traffic



Use GPU/TPU for faster predictions